

UNIVERSITÀ DEGLI STUDI DI SALERNO

Corso di Laurea in Ingegneria Informatica

Immagine da
inserire

images/logo
unisa.png

Anno Accademico 2025/2026

Relazione tecnica

IronHabit

TRACCIA PROGETTUALE 2 – FITNESS & WORKOUT TRACKER APP

Gruppo 9

Autori

Mario Di Gregorio

Gabriele Alfano

Angelo Ambrosio

Matricole

0612708906

0612710143

0612709750

Indice

1 Executive summary

IronHabit è un'applicazione mobile dedicata alla pianificazione, all'esecuzione e al monitoraggio degli allenamenti. Il progetto riunisce in un unico ambiente il catalogo degli esercizi, la creazione delle schede, la pianificazione settimanale delle sessioni, l'allenamento guidato, lo storico e gli obiettivi personali. L'app è pensata per un singolo utente locale, non richiede la creazione di un account, funziona senza backend e conserva i dati direttamente sul dispositivo.

Il prototipo è sviluppato con React Native, Expo e TypeScript. Il codice è organizzato in livelli sulla base di un'architettura progettuale, la Onion Architecture: i tipi e le entità del dominio sono separati dai casi d'uso, dalla persistenza e dall'interfaccia. Zustand raccoglie i dati condivisi tra le schermate e le relative operazioni; il middleware di persistenza salva in formato JSON esercizi, schede, sessioni, allenamenti, obiettivi, impostazioni e allenamento attivo tramite AsyncStorage per una memorizzazione locale.

L'app comprende una configurazione iniziale, un catalogo di esercizi di base, la costruzione di schede con serie adattive, la pianificazione delle sessioni e una modalità di allenamento guidato. Il workout attivo può essere ridotto e ripreso senza perdere i valori inseriti; il completamento di una serie avvia il timer di recupero. Lo storico permette di consultare e modificare gli allenamenti salvati, confrontarli nel tempo e creare una nuova scheda a partire da un workout precedente. La Home mostra un riepilogo compatto con volume settimanale, stato delle sessioni, numero di workout e obiettivi attivi.

2 Descrizione dell'app

2.1 Obiettivo dell'applicazione

L'obiettivo di IronHabit è ridurre la frammentazione tipica della gestione di un allenamento. Schede, calendario, recuperi, valori delle serie, storico e obiettivi non sono trattati come strumenti indipendenti: i dati inseriti durante la preparazione vengono riutilizzati nell'esecuzione e, al termine, alimentano lo storico e il calcolo dei progressi.

2.2 Tipologia di utenti

L'app è rivolta a persone che svolgono attività in palestra, a corpo libero o di tipo cardio e desiderano:

- organizzare un catalogo personale di esercizi;
- creare schede con serie, carichi, ripetizioni, tempi, distanze e recuperi;
- pianificare più sessioni nella stessa settimana;
- seguire un allenamento guidato e registrare i risultati effettivi;
- consultare record, confronti e obiettivi senza utilizzare servizi remoti.

L'app gestisce il profilo locale e non prevede ruoli amministrativi, funzioni social o account multipli.

2.3 Problema affrontato

Durante un allenamento l'utente deve ricordare la scheda, i valori della volta precedente, i tempi di recupero e gli obiettivi da raggiungere. Usare strumenti separati aumenta il rischio di perdere informazioni o di registrare informazioni sempre diverse. IronHabit collega invece un pianificazione all'esecuzione reale: una scheda può essere programmata in una data, trasformata in una bozza di workout, completata serie per serie e infine salvata nello storico. I dati registrati vengono poi utilizzati per aggiornare obiettivi, volume, record e confronti temporali.

2.4 Principali scenari d'uso

1. **Configurazione iniziale:** l'utente al primo avvio inserisce il proprio nome, sceglie tra i temi offerti dall'applicazione ed è possibile definire una prima configurazione del suono, della vibrazione e del tempo di predefinito.
2. **Gestione degli esercizi:** l'utente consulta il catalogo iniziale, ricerca e filtra gli elementi, tra le possibili opzioni offerte, oppure crea, modifica ed elimina esercizi personali.
3. **Creazione di una scheda:** l'utente seleziona più esercizi in ordine e configura, per ogni serie, tipo, ripetizioni, carico, durata, distanza e recupero in base alla modalità di tracciamento dell'esercizio.
4. **Pianificazione:** l'utente associa una scheda a un giorno oppure crea una sessione libera con titolo e note. Un vincolo importante in termini di pianificazione è: più sessioni possono essere inserite nello stesso giorno, ma la stessa scheda non può essere pianificata due volte nella medesima data.
5. **Esecuzione:** l'utente avvia un workout da una scheda, da una sessione pianificata o in modalità libera; modifica i valori delle serie e utilizza il timer automatico di recupero.
6. **Uso dell'app durante l'allenamento:** l'utente può lasciare temporaneamente la schermata del workout, consultare altre aree dell'app e poi tornare all'allenamento senza perdere i dati inseriti.
7. **Analisi:** l'utente consulta lo storico, modifica un workout salvato, confronta durata, ripetizioni e carico massimo con una sessione precedente e visualizza record e stima dell'1RM.
8. **Obiettivi:** l'utente crea obiettivi di frequenza settimanale, carico o ripetizioni e ne osserva l'avanzamento calcolato dai workout registrati.

3 Requisiti

3.1 Requisiti funzionali implementati

ID	Requisito	Descrizione	Priorità	Rischio
RF01	Gestione esercizi	L'utente può visualizzare, creare, modificare ed eliminare gli esercizi. Per ogni esercizio può inserire descrizione, muscoli coinvolti, difficoltà, attrezzatura, ripetizioni consigliate, durata, note e tipo di tracciamento.	1	Medio
RF02	Gestione schede	L'utente può visualizzare, creare, modificare ed eliminare le schede di allenamento. Può inoltre scegliere gli esercizi, ordinarli e impostare gli obiettivi delle singole serie.	1	Alto
RF03	Pianificazione sessioni	L'utente può organizzare gli allenamenti nell'agenda settimanale, creando, modificando o eliminando le sessioni. Una sessione può essere collegata a una scheda oppure essere libera, con titolo e note, e può risultare da svolgere, completata o saltata.	1	Medio
RF04	Registrazione workout	Un workout viene registrato dopo aver completato un allenamento guidato o libero. L'utente può consultarlo nello storico, modificarlo, confrontarlo con quelli precedenti e usarlo per creare una nuova scheda.	1	Alto
RF05	Gestione obiettivi	L'utente può visualizzare, creare, modificare ed eliminare obiettivi relativi alla frequenza settimanale, al carico o alle ripetizioni. I progressi vengono aggiornati automaticamente in base ai workout completati.	2	Medio
RF06	Ricerca e filtri	L'utente può effettuare ricerche, applicare filtri e ordinare gli elementi presenti nelle sezioni esercizi, schede, sessioni, storico e obiettivi.	2	Basso
RF07	Riepiloghi e analisi	La Home mostra un riepilogo dei workout, degli obiettivi attivi, del volume settimanale e dello stato delle sessioni. Nei dettagli sono presenti anche record, stima dell'1RM e confronti tra allenamenti.	2	Medio
RF08	Navigazione	L'app è divisa in cinque sezioni principali. Da queste è possibile raggiungere le schermate per inserire o modificare i dati, vedere i dettagli e aprire il workout a schermo intero.	1	Medio

ID	Requisito	Descrizione	Priorità	Rischio
RF09	Persistenza locale	I dati, le preferenze dell'utente e l'eventuale workout in corso vengono salvati sul dispositivo tramite Zustand e AsyncStorage, in modo da non perderli alla chiusura dell'app.	1	Alto
RF10	Allenamento guidato	L'utente può iniziare un allenamento da una scheda, da una sessione oppure in modalità libera. Durante il workout può gestire le serie, vedere i valori precedenti, usare il timer, aggiungere, rimuovere o riordinare gli elementi e riprendere l'allenamento senza perdere i dati.	1	Alto
RF11	Configurazione utente	Al primo avvio l'utente completa una configurazione iniziale. Può inserire il nome, scegliere il tema, attivare o disattivare suoni e vibrazione e impostare il tempo di recupero predefinito.	3	Basso
RF12	Catalogo iniziale	Al primo avvio sono già disponibili alcuni esercizi di base. Le schede, le sessioni, gli allenamenti nello storico e gli obiettivi vengono invece creati dall'utente.	3	Basso

3.2 Requisiti non funzionali

ID	Requisito	Descrizione	Priorità	Rischio
RNF01	Usabilità	L'app presenta una navigazione chiara, azioni principali facilmente riconoscibili e messaggi che avvisano l'utente quando i dati inseriti non sono corretti. Le schermate vuote indicano inoltre cosa fare per iniziare.	1	Medio
RNF02	Portabilità	Grazie a React Native ed Expo, il progetto può essere eseguito su Android, iOS e web, tenendo conto delle funzioni supportate dalle librerie utilizzate.	2	Medio
RNF03	Funzionamento offline	Le funzioni principali dell'app possono essere utilizzate senza connessione a Internet, perché non sono necessari servizi online, autenticazione o un server esterno.	1	Medio
RNF04	Integrità dei dati	Prima di salvare le informazioni, l'app controlla che i riferimenti, le date e i valori numerici inseriti siano validi.	1	Alto
RNF05	Manutenibilità	Il codice è diviso in parti con compiti diversi, come gestione dei dati, regole dell'app e interfaccia. Sono inoltre presenti componenti e funzioni condivise per evitare ripetizioni.	2	Medio
RNF06	Reattività	Quando un dato cambia, vengono aggiornate solo le parti della schermata che ne hanno bisogno. Filtri e statistiche vengono calcolati direttamente sul dispositivo.	2	Medio
RNF07	Accessibilità di base	Le azioni principali hanno etichette chiare e aree di tocco adeguate. L'interfaccia utilizza un buon contrasto e permette di configurare i feedback sonori e la vibrazione.	3	Medio

3.3 Feature avanzate scelte

Le feature avanzate integrate nell'app sono:

- **Allenamento sempre disponibile:** quando un workout è in corso, l'utente può uscire dalla schermata, usare altre parti dell'app e poi tornare all'allenamento senza perdere i dati inseriti.
- **Timer di recupero:** dopo il completamento di una serie parte automaticamente un conto alla rovescia. Il tempo di recupero può essere impostato tra 30 e 180 secondi e, durante il timer, è possibile aggiungere altri 30 secondi.

- **Serie adattate al tipo di esercizio:** i campi da compilare cambiano in base all'esercizio scelto. Quando disponibile, l'app mostra anche i valori inseriti nello stesso esercizio durante l'allenamento precedente.
- **Modifica del workout durante l'allenamento:** durante la sessione l'utente può aggiungere, eliminare o riordinare esercizi e serie. Alla fine può anche scegliere di salvare queste modifiche nella scheda di partenza.
- **Creazione di una scheda dallo storico:** un allenamento già completato può essere usato come base per creare una nuova scheda, che può poi essere modificata.
- **Controllo dei progressi:** l'app calcola alcuni dati utili, come il volume totale, il carico massimo raggiunto, i record personali e una stima del carico massimo eseguibile per una ripetizione.

3.4 Funzionalità considerate ma non implementate completamente

- eliminazione di un workout già presente nello storico;
- selezione preventiva degli esercizi per una sessione libera pianificata;
- configurazione completa di superset e RPE direttamente nel live workout: tali dati sono previsti nel modello e modificabili nello storico, ma non sono esposti in ogni fase del flusso;
- generazione automatica delle serie di riscaldamento e calcolo dei dischi: sono presenti funzioni di supporto nel codice, ma non sono collegate alle schermate e non vengono considerate feature consegnate;
- autenticazione, sincronizzazione cloud, backup automatico, condivisione sociale, notifiche push e integrazione con smartwatch o sensori.

3.5 Limitazioni note

- Il profilo è singolo e locale, la disinstallazione o la cancellazione dei dati dell'app elimina l'archivio dei dati;
- Non esiste una sincronizzazione tra dispositivi, né un'esportazione o backup automatico;
- Lo storico non espone ancora il comando per eliminare definitivamente un workout salvato;
- Una sessione libera viene pianificata con titolo e note, mentre gli esercizi vengono scelti dopo l'avvio del workout;
- Il timer è affidabile durante l'uso dell'app, ma non è progettato come servizio nativo di qui è garantito il funzionamento in background;
- Le date e orari dipendono dalle impostazioni del dispositivo e non sono gestiti scenari multi-fus orari;
- Le statistiche sono interne all'applicazione e non costituiscono indicazioni mediche o prescrizioni di allenamento;

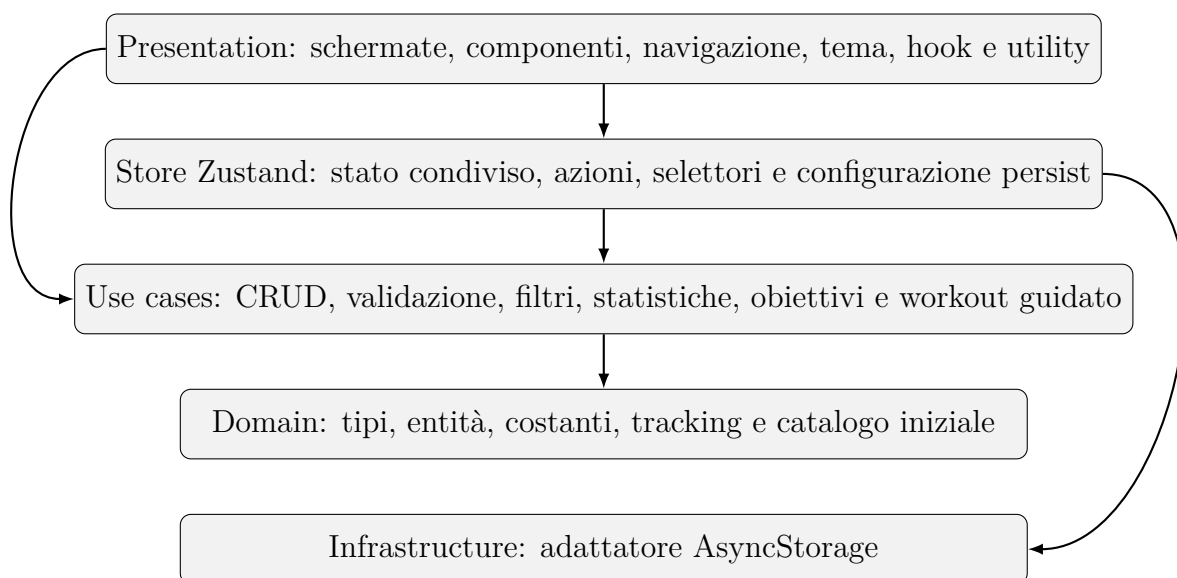
- Non sono ancora stati realizzati test automatici completi per controllare il funzionamento del codice e dell'interfaccia.

4 Progettazione dell'app

4.1 Struttura generale

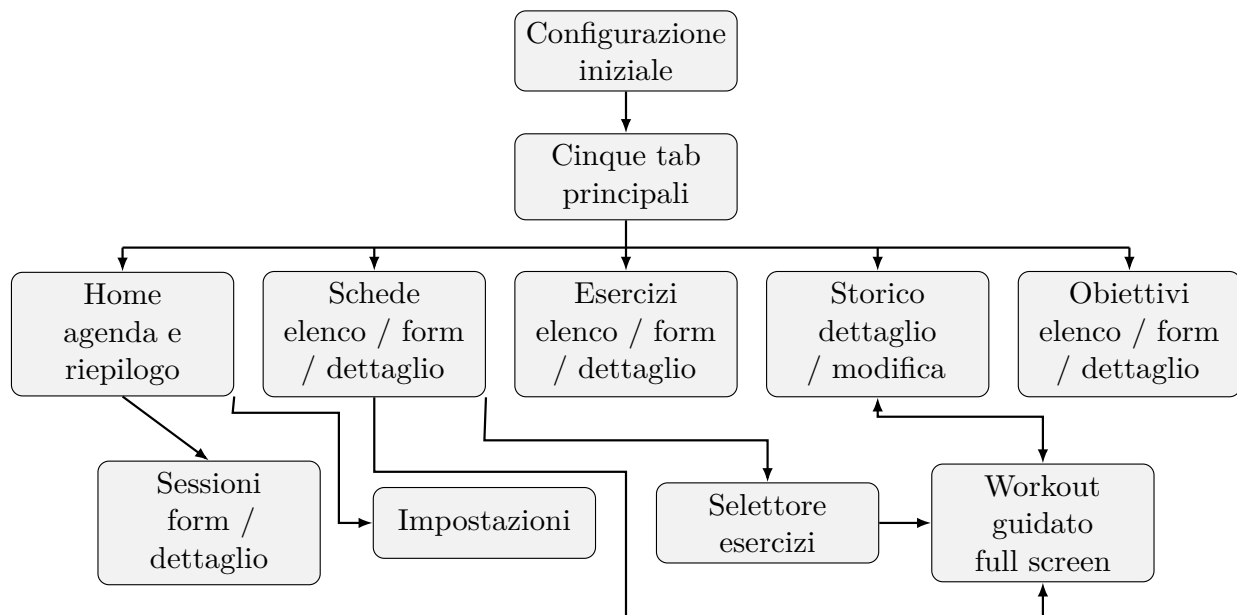
Al primo avvio viene mostrata la schermata di configurazione iniziale. L'app offre un'esperienza organizzata intorno a cinque tab principali: **Home**, **Schede**, **Esercizi**, **Storico** e **Obiettivi**. Le impostazioni, i form, le viste di dettaglio e la pianificazione delle sessioni appartengono allo stack di navigazione. Il workout guidato e il selettore multiplo degli esercizi utilizzano modali a schermo intero.

4.2 Architettura logica



L'organizzazione è ispirata alla Onion Architecture. Le schermate modificano i dati attraverso le azioni dello store; alcune funzioni pure di filtro e statistica vengono richiamate direttamente dalla presentazione. Le operazioni principali dell'app utilizzano i dati definiti nel dominio. Il salvataggio locale è invece gestito da una parte separata del progetto, che si occupa di comunicare con AsyncStorage.

4.3 Flusso di navigazione



4.4 Organizzazione dei dati

Le entità principali sono:

- **Exercise**: nome, istruzioni, muscoli, difficoltà, attrezzatura, valori consigliati, tracking e note;
- **WorkoutPlan**: dati generali della scheda e lista ordinata di **PlanExercise**, ciascuno con target per serie;
- **PlannedSession**: data, titolo, eventuale **planId**, stato e note;
- **WorkoutLog**: data, durata, fatica, riferimenti opzionali a scheda e sessione, esercizi svolti e serie effettive;
- **Goal**: categoria, target, avanzamento, periodo, stato ed eventuale esercizio collegato;
- **ActiveWorkout**: bozza temporanea del workout con timestamp iniziale, stato di minimizzazione e scadenza del timer;
- **UserSettings**: nome del profilo, tema, recupero predefinito, feedback sonoro/tattile e completamento dell'onboarding.

Gli identificativi collegano esercizi, schede, sessioni, workout e obiettivi senza duplicare intere entità. Le eliminazioni di esercizi, schede e sessioni vengono bloccate quando produrrebbero riferimenti non validi. I dati derivati, come volume e statistiche, sono ricalcolati e non vengono salvati come copie permanenti.

5 Design dell'app

5.1 Obiettivo del design

Il design di IronHabit è stato pensato per rendere l'app semplice da utilizzare anche durante un allenamento. Le informazioni principali sono state organizzate in sezioni chiare, evitando schermate troppo cariche e mantenendo le azioni più importanti facilmente raggiungibili.

L'interfaccia segue uno stile coerente in tutte le aree dell'app. Titoli, pulsanti, card, campi di input e indicatori mantengono la stessa impostazione grafica, così l'utente può capire più velocemente come muoversi tra le schermate. È stata inoltre considerata la leggibilità sia con il tema chiaro sia con quello scuro.

5.2 Logo dell'app

Il logo di IronHabit richiama in modo diretto il tema dell'allenamento e della forza fisica. La figura centrale rappresenta un atleta con un bilanciere, mentre la forma circolare rossa mette in evidenza il soggetto e riprende uno dei colori utilizzati per gli elementi più importanti dell'interfaccia.

La scelta del bianco e del nero mantiene il logo semplice e facilmente riconoscibile, mentre il rosso aggiunge contrasto senza rendere l'immagine troppo complessa. Gli angoli arrotondati permettono inoltre di utilizzare il logo come icona dell'app sui dispositivi mobili.



5.3 Wireframe iniziali

Prima dello sviluppo dell'applicazione sono stati realizzati dei wireframe per definire la posizione dei contenuti, la struttura della navigazione e le azioni principali. I wireframe sono serviti come base iniziale e sono stati successivamente adattati in base alle necessità emerse durante lo sviluppo.

Immagine da inserire

images/schermate/Configurazione
Iniziale.png

Immagine da inserire

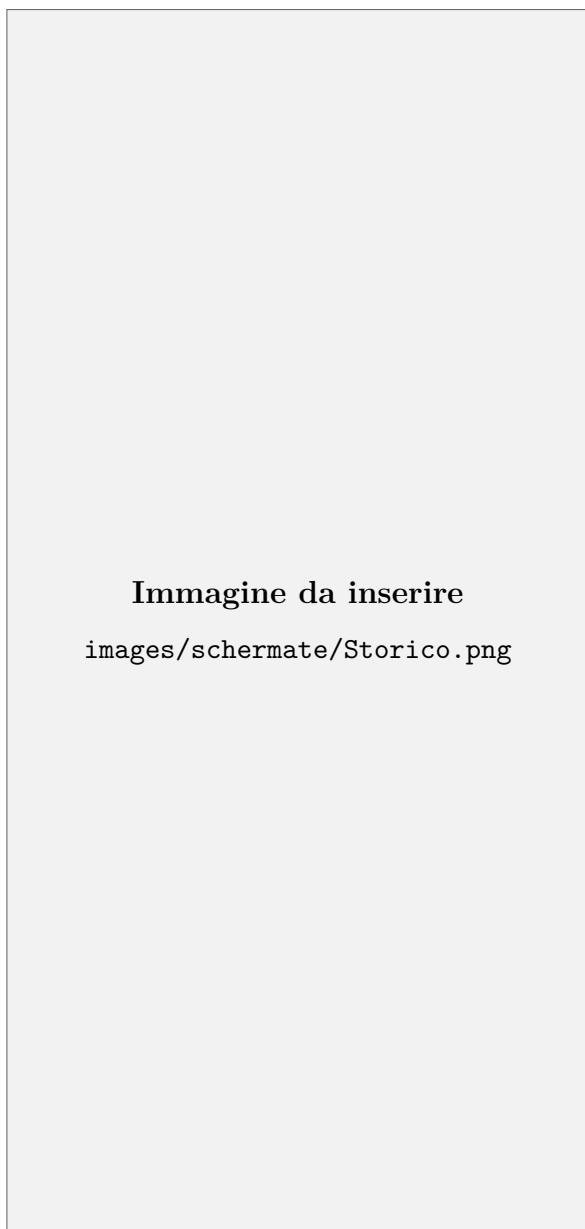
images/schermate/Home.png

Immagine da inserire

`images/schermate/Schede.png`

Immagine da inserire

`images/schermate/Esercizi.png`



5.4 Scelta dei colori e del tema

La scelta dei colori è partita da una base neutra composta principalmente da bianco, nero e diverse tonalità di grigio. Questa soluzione mantiene l'interfaccia pulita e permette di mettere in evidenza soltanto gli elementi importanti.

Nel tema chiaro lo sfondo principale è bianco, mentre card, bordi e aree secondarie utilizzano grigi molto chiari. Nel tema scuro vengono utilizzati sfondi quasi neri e testi chiari, in modo da mantenere un buon contrasto. L'utente può scegliere il tema chiaro, quello scuro oppure utilizzare automaticamente il tema impostato sul dispositivo.

Tema	Elemento	Colore
Chiaro	Sfondo principale	#FFFFFF
Chiaro	Sfondo secondario	#F5F5F5
Chiaro	Testo principale	#000000
Chiaro	Testo secondario	#666666
Chiaro	Bordi	#E0E0E0
Scuro	Sfondo principale	#09090B
Scuro	Sfondo secondario	#18181B
Scuro	Card	#111113
Scuro	Testo principale	#F7F7F8
Scuro	Testo secondario	#A1A1AA
Scuro	Bordi	#2A2A2E

I colori verde, blu, arancione e rosso vengono utilizzati soltanto quando hanno un significato preciso. Il verde indica risultati positivi o operazioni completate, il blu viene usato per alcune informazioni, l'arancione segnala situazioni da controllare e il rosso è riservato agli errori e alle azioni di eliminazione.

5.5 Scelta del font e organizzazione dei testi

L'app non utilizza un font esterno, ma mantiene il carattere predefinito del dispositivo. Questa scelta permette di avere una buona leggibilità e un aspetto coerente con il sistema operativo su cui viene eseguita l'app.

Le diverse parti del testo vengono distinte attraverso dimensione e spessore. I titoli delle schermate sono più grandi e marcati, mentre descrizioni, etichette e informazioni secondarie utilizzano dimensioni più contenute. In questo modo l'utente può riconoscere subito le informazioni principali senza rendere la schermata troppo pesante.

5.6 Stile dell'interfaccia

L'interfaccia utilizza card, pulsanti, input, selettori, filtri e indicatori con uno stile comune. Gli angoli sono leggermente arrotondati e gli spazi tra gli elementi sono mantenuti regolari, in modo da separare chiaramente i vari contenuti.

Le azioni principali sono mostrate con pulsanti ben visibili, mentre le operazioni secondarie utilizzano pulsanti più semplici o icone. Le icone mantengono lo stesso stile nelle diverse schermate e vengono usate soprattutto nella barra di navigazione e nelle azioni più frequenti.

Le schermate rispettano la safe area del dispositivo, permettono lo scorrimento quando i contenuti sono numerosi e si adattano a dimensioni dello schermo differenti. Su schermi più larghi viene inoltre limitata la larghezza dei contenuti, evitando che testi, form e card risultino troppo estesi.

5.7 Schermate finali dell'app



Immagine da inserire

images/schermate-finali/Schede.png

Immagine da inserire

images/schermate-finali/Esercizi.png

Immagine da inserire

images/schermate-finali/Storico.png

Immagine da inserire

images/schermate-finali/Obiettivo.png



5.8 Passaggio dai wireframe all'app finale

I wireframe hanno definito la struttura iniziale delle schermate e la posizione dei contenuti principali. Durante lo sviluppo sono stati aggiunti gli elementi necessari per rendere l'interfaccia realmente utilizzabile, come filtri, azioni, messaggi di controllo, grafici, agenda settimanale e navigazione tra le varie sezioni.

La struttura generale è rimasta vicina all'idea iniziale, ma alcuni elementi sono stati modificati per migliorare la leggibilità e adattarsi meglio ai dati reali. Sono stati inoltre aggiunti il tema chiaro e scuro, il timer di recupero, il mini-player del workout e i diversi stati delle sessioni e degli obiettivi.

6 Scelte tecnologiche

Tecnologia	Ruolo e motivazione
React Native 0.81.5	Utilizzato per sviluppare l'app mobile usando componenti riutilizzabili tra le diverse schermate. Permette inoltre di eseguire lo stesso progetto su dispositivi e sistemi operativi differenti.
Expo SDK 54	Utilizzato per avviare e provare facilmente il progetto tramite Expo Go, senza dover configurare subito un ambiente di sviluppo nativo completo. Permette anche di usare in modo semplice alcune funzionalità del dispositivo.
TypeScript 5.9	Utilizzato per definire in modo più chiaro i tipi dei dati, degli input e delle funzioni. Aiuta a individuare più facilmente eventuali errori durante lo sviluppo.
React Navigation 7	Utilizzato per gestire il passaggio tra le schermate dell'app, la barra di navigazione inferiore, le schermate di dettaglio, le modali e il passaggio dei dati.
Zustand 5	Utilizzato per gestire in un unico punto i dati condivisi tra le varie schermate, come esercizi, schede, obiettivi e allenamenti. Permette inoltre di aggiornare facilmente lo stato dell'app.
AsyncStorage 2.2	Utilizzato per salvare i dati direttamente nella memoria del dispositivo. In questo modo le informazioni rimangono disponibili anche dopo la chiusura dell'app.
NativeWind 4 / Tailwind CSS 3.4	Utilizzati per rendere più semplice e ordinata la gestione dello stile delle schermate, attraverso classi riutilizzabili affiancate agli stili standard di React Native.
Lucide React Native	Utilizzato per inserire icone semplici e coerenti nei pulsanti, nelle azioni e nella barra di navigazione.
react-native-svg 15.12	Utilizzato per realizzare grafici e indicatori direttamente nell'app, senza dover usare immagini già pronte.
expo-audio / expo-haptics	Utilizzati per fornire un feedback sonoro o tramite vibrazione durante alcune azioni, ad esempio al termine di una serie o del timer di recupero.
React Native Reanimated / Gesture Handler	Utilizzati per gestire alcune animazioni e interazioni dell'interfaccia, necessarie anche per il corretto funzionamento della navigazione e di alcuni componenti grafici.

La scelta di React Native con Expo riduce la configurazione necessaria per la demo su dispositivi Android e iOS. TypeScript rende più espliciti i dati del dominio, mentre Zustand evita di distribuire lo stato condiviso in numerosi componenti. AsyncStorage è adeguato perché il volume dei dati del prototipo è contenuto e non sono richieste query relazionali complesse.

6.1 Modalità di persistenza

La persistenza dei dati viene gestita tramite il middleware `persist` di Zustand, che utilizza `createJSONStorage` insieme ad `AsyncStorage`. Zustand si occupa di scegliere e convertire i dati da salvare in formato JSON, mentre `AsyncStorage` li memorizza nella memoria locale del dispositivo.

La funzione `partialize` permette di salvare soltanto i dati che devono rimanere disponibili dopo la chiusura dell'app: esercizi, schede, sessioni pianificate, workout, obiettivi, impostazioni e workout attivo. Non vengono invece salvati i metodi dello store, i selettori e gli stati temporanei utilizzati durante alcune operazioni.

La chiave utilizzata per il salvataggio è `ironhabit-storage-v4`. Questa soluzione è adatta a un'app locale e utilizzata da un singolo utente. Nel caso di una quantità maggiore di dati, ricerche più complesse, sincronizzazione tra dispositivi o accesso da parte di più utenti, sarebbe più adatto utilizzare un database e un servizio remoto.

7 Implementazione

7.1 Organizzazione del codice

Percorso	Responsabilità
<code>src/domain/</code>	Contiene i tipi e le entità principali dell'app, le costanti, le modalità di registrazione degli esercizi e il catalogo iniziale.
<code>src/usecases/</code>	Contiene le operazioni di creazione, modifica ed eliminazione dei dati, i controlli sugli input, i filtri, le statistiche, l'aggiornamento degli obiettivi e la logica del workout guidato.
<code>src/infrastructure/</code>	Contiene il componente che permette di salvare i dati localmente tramite AsyncStorage.
<code>src/presentation/store/</code>	Contiene lo store Zustand, le funzioni che modificano lo stato, i dati messi a disposizione delle schermate e la configurazione del salvataggio automatico.
<code>src/presentation/navigation/</code>	Contiene la navigazione tra le sezioni principali, le schermate di dettaglio, le finestre a schermo intero, il mini-player e il passaggio dei parametri.
<code>src/presentation/screens/</code>	Contiene le schermate dell'app, come elenchi, form, dettagli, dashboard, configurazione iniziale e workout guidato.
<code>src/presentation/components/</code>	Contiene i componenti grafici riutilizzabili, come filtri, indicatori, grafici, timer e messaggi mostrati quando non sono presenti dati.
<code>src/presentation/hooks/</code>	Contiene la logica riutilizzabile per la gestione del conto alla rovescia.
<code>src/presentation/theme/</code>	Contiene i colori e gli stili usati per il tema chiaro, scuro o impostato in base al sistema.
<code>src/presentation/utils/</code>	Contiene funzioni di supporto per date, testi, avvisi, audio e gestione degli stili.

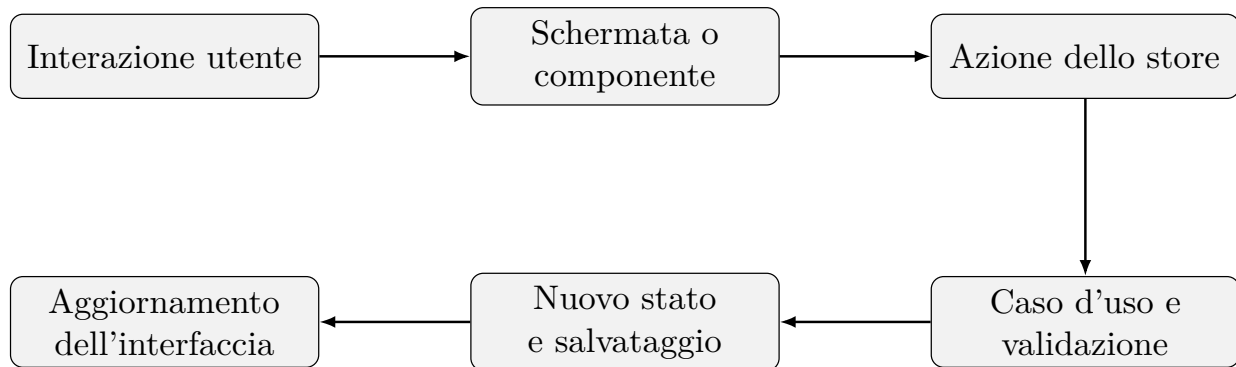
7.2 Componenti principali

I componenti più ricorrenti sono:

- `ScreenContainer`, `PageHeader` e `Card` per la struttura comune delle schermate;
- `AppButton`, `IconButton`, `AppInput`, `AppSelect`, `EntitySelect` e `MultiSelect` per input e azioni coerenti;
- `ListRow`, `EmptyState` e `FiltersPanel` per elenchi, assenza di dati e filtri richiudibili;
- `RestTimerSelect`, `TimerPanel` e `Checkbox` per le interazioni del workout;
- `MetricStrip`, `ProgressBar` e `AnalyticsOverview` per riepiloghi e grafici;
- `ThemeProvider` per applicare il tema scelto e `AppNavigator` per coordinare tab, stack e modali.

7.3 Gestione dello stato e passaggio dei dati

Le schermate leggono dallo store soltanto le porzioni necessarie e invocano azioni come `addPlan`, `updateSession`, `startWorkout` o `completeActiveWorkout`. Le azioni applicative richiamano i casi d'uso, che validano l'input e restituiscono il nuovo stato. Il middleware di Zustand serializza automaticamente le parti configurate e le schermate sottoscritte vengono aggiornate.



7.4 Gestione della navigazione

La navigazione dell'app utilizza principalmente due tipi di navigazione: la **tab navigation** e la **stack navigation**. La tab navigation è rappresentata dalla barra inferiore e permette di passare rapidamente tra le cinque sezioni principali dell'app: Home, Schede, Esercizi, Storico e Obiettivi.

La stack navigation viene invece utilizzata per aprire schermate successive, come i form di inserimento, le pagine di dettaglio e le impostazioni. In questo caso le schermate vengono aggiunte una sopra l'altra e l'utente può tornare indietro alla schermata precedente. Non è stata utilizzata una drawer navigation, perché l'app non presenta un menu laterale.

Il workout guidato e il selettore degli esercizi vengono aperti come schermate a tutto schermo. Per passare i dati da una schermata all'altra non viene inviato l'intero oggetto, ma soltanto il suo identificativo. Ad esempio, per aprire il dettaglio di un allenamento viene passato il suo `id`; la schermata recupera poi i dati aggiornati dallo store. In questo modo si evita di mostrare una versione vecchia dei dati dopo una modifica.

Quando l'utente lascia temporaneamente la schermata del workout, l'allenamento non viene interrotto. I suoi dati rimangono salvati nello store tramite `activeWorkout` e viene mostrato un mini-player sopra la barra di navigazione. Da questo elemento l'utente può riaprire il workout e continuare da dove aveva lasciato.

7.5 Validazione e integrità

Prima di salvare o modificare i dati, l'app controlla che le informazioni inserite siano corrette e coerenti. In particolare:

- una scheda deve contenere almeno un esercizio valido;

- una sessione deve essere collegata a una scheda esistente oppure, se si tratta di una sessione libera, deve avere almeno un titolo;
- la stessa scheda non può essere programmata due volte nello stesso giorno;
- le sessioni non svolte e con una data già passata vengono indicate automaticamente come saltate;
- un esercizio, una scheda o una sessione non possono essere eliminati se sono ancora utilizzati in altre parti dell'app;
- un workout deve contenere esercizi e serie con valori adatti al tipo di attività, ad esempio carico e ripetizioni oppure durata e distanza;
- il valore RPE, usato per indicare lo sforzo percepito, deve essere compreso tra 0 e 10;
- per gli obiettivi legati al carico o alle ripetizioni è necessario scegliere un esercizio e la data di scadenza non può essere precedente alla data di inizio.

7.6 Allenamento guidato

Quando l'utente avvia un allenamento da una scheda, l'app crea una bozza chiamata `ActiveWorkout`. Gli esercizi e le serie presenti nella scheda vengono copiati all'interno della bozza e inizialmente risultano ancora da completare. I campi mostrati cambiano in base al tipo di esercizio: ad esempio, per alcuni vengono richiesti carico e ripetizioni, mentre per altri durata o distanza.

Se l'utente avvia una sessione libera, viene invece creato un workout vuoto, al quale può aggiungere gli esercizi scegliendoli dal catalogo.

Quando una serie viene completata, l'app aggiorna il workout in corso, attiva il suono o la vibrazione scelti nelle impostazioni e avvia il timer di recupero. L'istante in cui il timer deve terminare viene salvato, così il tempo rimanente può essere calcolato nuovamente anche dopo aver lasciato temporaneamente la schermata.

Durante l'allenamento l'utente può modificare i valori delle serie e le note, cambiare il tipo di serie, aggiungere o eliminare serie, inserire nuovi esercizi e modificarne l'ordine.

La durata dell'allenamento viene calcolata al momento del salvataggio confrontando l'orario di inizio con quello di completamento:

Listing 1: Calcolo della durata effettiva al completamento

```
const durationMinutes = Math.max(  
  1,  
  Math.round((Date.now() - activeWorkout.startedAt) / 60000)  
);
```

Se l'allenamento è stato avviato da una scheda e durante la sessione sono stati modificati gli esercizi, il loro ordine, i tempi di recupero o i valori delle serie, l'app confronta il workout completato con la scheda originale. Al termine, l'utente può scegliere se applicare le modifiche anche alla scheda oppure salvarle soltanto nello storico dell'allenamento.

7.7 Statistiche e indicatori

Il volume di un workout viene calcolato sommando, per ogni serie completata, il prodotto tra il carico utilizzato e il numero di ripetizioni:

$$volume = carico \times ripetizioni$$

Per stimare il carico massimo che l'utente potrebbe sollevare per una sola ripetizione viene utilizzata la formula di Epley:

$$1RM = carico \cdot \left(1 + \frac{ripetizioni}{30}\right)$$

L'app calcola anche altri dati, come la durata totale degli allenamenti, la frequenza media, il numero di settimane consecutive con almeno un workout, la distribuzione delle serie tra i vari gruppi muscolari, l'andamento dei carichi e il volume delle ultime otto settimane.

Nella versione attuale, la Home mostra le informazioni principali: numero di workout completati, obiettivi attivi, grafico del volume settimanale e riepilogo delle sessioni da svolgere, completate o saltate. Nelle schermate di dettaglio degli esercizi e degli allenamenti vengono invece mostrati i record raggiunti, la stima dell'1RM e il confronto con i risultati precedenti.

7.8 Ricerca, filtri e ordinamento

Le operazioni di ricerca e filtro sono gestite da funzioni separate dalle schermate. Queste funzioni leggono i dati disponibili e restituiscono solamente gli elementi che rispettano i criteri scelti, senza modificare i dati salvati.

La ricerca non fa differenza tra lettere maiuscole e minuscole. Gli esercizi possono essere cercati o filtrati per nome, gruppo muscolare, difficoltà e attrezzatura. Le schede possono essere filtrate per obiettivo e livello, le sessioni per stato, lo storico per periodo e gli obiettivi per categoria e stato.

L'utente può inoltre ordinare i risultati in modi diversi in base alla sezione, ad esempio per data, nome, durata o percentuale di avanzamento.

7.9 Gestione dei dati persistenti

La configurazione dei dati salvati localmente è la seguente:

Listing 2: Dati inclusi nella persistenza locale

```
partialize: (state) => ({
  exercises: state.exercises,
  plans: state.plans,
  plannedSessions: stripLegacySessionTypes(
    state.plannedSessions
  ),
  workoutLogs: state.workoutLogs,
  goals: state.goals,
```

```
settings: state.settings,  
activeWorkout: state.activeWorkout  
})
```

La funzione `partialize` indica quali parti dello store devono essere salvate tramite AsyncStorage. Vengono conservati gli esercizi, le schede, le sessioni pianificate, gli allenamenti completati, gli obiettivi, le impostazioni e l'eventuale workout ancora in corso.

Prima del salvataggio, la funzione `stripLegacySessionTypes` rimuove dalle sessioni pianificate un vecchio campo non più utilizzato. In questo modo vengono conservati soltanto i dati compatibili con la struttura attuale dell'applicazione.

I dati che possono essere calcolati a partire da queste informazioni, come grafici, totali e statistiche, non vengono salvati direttamente. Vengono ricalcolati quando servono, evitando di memorizzare più volte le stesse informazioni.

7.10 Parti più significative e difficoltà incontrate

Le parti che hanno richiesto maggiore attenzione durante lo sviluppo sono state:

- mantenere attivo il workout anche quando l'utente lascia temporaneamente la schermata, permettendogli di tornare all'allenamento senza perdere i dati inseriti;
- gestire esercizi diversi tra loro, alcuni basati su carico e ripetizioni e altri su durata o distanza, evitando di creare form completamente separati;
- mantenere corretti i collegamenti tra esercizi, schede, sessioni, allenamenti e obiettivi, soprattutto durante le operazioni di modifica o eliminazione;
- gestire correttamente le date delle sessioni pianificate, utilizzando il giorno locale del dispositivo ed evitando errori dovuti alla conversione automatica dell'orario;
- aggiornare automaticamente lo stato delle sessioni e il progresso degli obiettivi quando viene completato un allenamento;
- rendere l'interfaccia leggibile sia con il tema chiaro sia con quello scuro e adattarla a dispositivi con dimensioni dello schermo differenti.

8 Verifica del prototipo

8.1 Controlli eseguiti

Sul progetto sono stati eseguiti i seguenti controlli tecnici:

- `npm run typecheck`: controllo del codice TypeScript senza generazione di nuovi file e senza errori;
- `npx tsc -noEmit -noUnusedLocals -noUnusedParameters`: controllo aggiuntivo per individuare variabili, import e parametri non utilizzati;

- `npm run doctor`: verifica della configurazione generale del progetto Expo;
- `npx expo install -check`: controllo della compatibilità tra Expo e le dipendenze installate;
- `npx expo start -lan -c`: avvio del server Metro in modalità LAN con pulizia della cache;
- avvio e prova dell'app su un dispositivo fisico tramite Expo Go.

8.2 Piano di collaudo manuale

Flusso	Esito atteso	Stato documentato
Configurazione iniziale	Nome e preferenze vengono salvati e si accede ai tab principali.	Verificato
CRUD esercizi	Inserimento, dettaglio, modifica ed eliminazione coerenti con i vincoli.	Verificato
CRUD schede	Creazione con esercizi e serie, modifica, dettaglio ed eliminazione.	Verificato
Pianificazione	Inserimento di più sessioni, blocco dei duplicati e aggiornamento degli stati.	Verificato
Workout guidato	Serie, timer, valori precedenti, minimizzazione, ripresa e salvataggio.	Verificato
Persistenza	Dati e workout attivo disponibili dopo la riapertura dell'app.	Verificato
Obiettivi	Avanzamento aggiornato dopo il salvataggio di un workout coerente.	Verificato
Storico	Ricerca, dettaglio, modifica, confronto e creazione scheda.	Verificato
Eliminazione workout salvato	Il workout dovrebbe essere rimosso mantenendo coerenti sessioni e obiettivi.	Non implementata
Test automatici	Copertura sistematica dei casi d'uso e dei componenti.	Non presenti

9 Istruzioni per l'esecuzione

9.1 Prerequisiti

Per eseguire il progetto sono necessari Node.js LTS, npm ed Expo Go installato su un dispositivo Android o iOS. Non è richiesta l'installazione globale di Expo e non sono necessari un backend o un database esterno.

Per utilizzare Expo Go, il computer e il dispositivo mobile devono essere collegati alla stessa rete.

9.2 Installazione e avvio

Dopo aver scaricato o clonato il repository, accedere alla cartella principale del progetto ed eseguire:

```
cd IronHabit
npm run setup
npm run start
```

Il comando `npm run setup` rimuove eventuali dipendenze già presenti e le installa nuovamente. Il comando `npm run start` avvia il server di sviluppo Expo.

Quando nel terminale viene mostrato il QR code, è possibile scansionarlo tramite Expo Go per aprire l'app sul dispositivo mobile.

Negli avvii successivi, se le dipendenze sono già installate, è sufficiente eseguire:

```
cd IronHabit
npm run start
```

9.3 Comandi aggiuntivi

Per avviare Expo eliminando la cache precedente:

```
npx expo start -c
```

Se la connessione tramite rete locale non funziona, è possibile utilizzare la modalità tunnel:

```
npx expo start --tunnel -c
```

Per controllare il codice TypeScript:

```
npm run typecheck
```

Per controllare la configurazione e le dipendenze del progetto Expo:

```
npm run doctor
```

Per avviare il progetto su un emulatore Android:

```
npm run android
```

Per eseguire la versione web:

```
npm run web
```

9.4 Repository

Repository del gruppo:

<https://github.com/digregoriomario/IronHabit>

10 Conclusioni e sviluppi futuri

IronHabit realizza un flusso coerente che parte dalla definizione degli esercizi e arriva alla pianificazione, all'esecuzione e all'analisi degli allenamenti. La separazione tra dominio, casi d'uso, persistenza e presentazione rende il progetto comprensibile e modificabile. La persistenza locale consente una demo offline, mentre il workout guidato, il timer e la minimizzazione costituiscono feature avanzate realmente integrate nel funzionamento quotidiano dell'app.

Successivamente si potranno integrare nell'interfaccia superset, RPE, riscaldamenti automatici e calcolo dei dischi, si potrebbe pensare ad aggiungere una suite di test, esportazione e backup, notifiche locali e un'eventuale sincronizzazione cloud mantenendo una modalità offline affidabile.